

Архитектура ОС UNIX

Санкт-Петербургский государственный политехнический университет

13 октября 2011

Сокет (socket) - это конечная точка сетевых коммуникаций. Он является чем-то вроде «портала», через которое можно отправлять байты во внешний мир. Приложение просто пишет данные в сокет, а их дальнейшая буферизация, отправка и транспортировка осуществляется используемым стеком протоколов и сетевой аппаратурой. Чтение данных из сокета происходит аналогичным образом.

В программе сокет идентифицируется дескриптором - это просто переменная типа `int`. Программа получает дескриптор от операционной системы при создании сокета, а затем передаёт его сервисам `socket API` для указания сокета, над которым необходимо выполнить то или иное действие.

С каждым сокет связываются три атрибута:

- домен
- тип
- протокол

Для создания сокета используется функция `socket`, имеющая следующий прототип:

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
int socket(int domain, int type, int protocol);
```

- PF_UNIX, PF_LOCAL
- PF_INET
- PF_INET6
- PF_IPX
- PF_NETLINK
- PF_X25
- PF_APPLETALK
- PF_PACKET

Тип сокета определяет способ передачи данных по сети.

- SOCK_STREAM
- SOCK_DGRAM
- SOCK_SEQPACKET
- SOCK_RAW
- SOCK_RDM
- SOCK_PACKET

Атрибут — определяет протокол, используемый для передачи данных. Как правило одозначно определяется по домену и тогда в качестве параметра можно указать 0, что соответствует протоколу по умолчанию, если же нет то можно посмотреть в protocols.

Прежде чем передавать данные через сокет, его необходимо связать с адресом в выбранном домене (именование сокета). Вид адреса зависит от выбранного домена.

В Unix-домене это текстовая строка - имя файла, через который происходит обмен данными. В Internet-домене адрес задаётся комбинацией IP-адреса и 16-битного номера порта.

Для явного связывания сокета с некоторым адресом используется функция `bind`. Её прототип имеет вид:

```
#include <sys/types.h>  
#include <sys/socket.h>
```

```
int bind(int sockfd, struct sockaddr *addr, int  
        addrlen);
```

В качестве первого параметра передаётся дескриптор сокета, который нужно привязать к заданному адресу. Второй параметр, `addr`, содержит указатель на структуру с адресом, а третий - длину этой структуры.

```
struct sockaddr {  
    unsigned short    sa_family;    // Семейство  
        адресов , AF_xxx  
    char              sa_data[14];  // 14 байтов  
        для хранения адреса  
};
```

Поле `sa_family` содержит идентификатор домена.

Работать на прямую со структурой sockaddr не удобно, поэтому для каждого домена существуют вспомогательные структу, такие как: sockaddr_in (от Internet), sockaddr_un (от Unix).

```
struct sockaddr_in {  
    short int          sin_family;  // Семейство  
        адресов  
    unsigned short int sin_port;    // Номер порта  
    struct in_addr      sin_addr;    // IP-адрес  
    unsigned char      sin_zero[8]; // "Дополнение"  
        до размера структуры sockaddr  
};
```

Здесь поле sin_family соответствует полю sa_family в sockaddr, в sin_port записывается номер порта, а в sin_addr - IP-адрес хоста.

Поле `sin_addr` само является структурой, которая имеет вид:

```
struct in_addr {  
    unsigned long s_addr;  
};
```

Оставлено в виде структуры для обратной совместимости (раньше был union).

Порядок байт при передаче адреса

Существует два порядка хранения байтов в слове и двойном слове. Один из них называется порядком хоста (host byte order), другой - сетевым порядком (network byte order) хранения байтов.

При указании IP-адреса и номера порта необходимо преобразовать число из порядка хоста в сетевой. Для этого используются функции `htons` (Host TO Network Short) и `htonl` (Host TO Network Long). Обратное преобразование выполняют функции `ntohs` и `ntohl`.

Установка соединения (сервер)

Установка соединения на стороне сервера состоит из четырёх этапов, ни один из которых не может быть опущен.

Сначала сокет создаётся и привязывается к локальному адресу, можно либо выбрать один из IP адресов интерфейсов, либо задать константу `INADDR_ANY`. Номер порта задается числом, либо 0, во втором случае система сама выберет случайный порт.

Установка соединения (сервер)

На следующем шаге создаётся очередь запросов на соединение. При этом сокет переводится в режим ожидания запросов со стороны клиентов. Всё это выполняет функция `listen`:

```
int listen(int sockfd, int backlog);
```

Первый параметр - дескриптор сокета, а второй задаёт размер очереди запросов. Если очередь заполнена, все последующие запросы будут игнорироваться.

Установка соединения (сервер)

Когда сервер готов обслуживать очередной запрос, он вызывает функцию `accept`:

```
#include <sys/socket.h>
```

```
int accept(int sockfd, void *addr, int *addrlen);
```

Установка соединения (сервер)

Функция `accept` создаёт для общения с клиентом новый сокет и возвращает его дескриптор.

Параметр `sockfd` задаёт слушающий сокет.

В структуру, на которую ссылается `addr`, записывается адрес сокета клиента, который установил соединение с сервером.

В переменную, адресуемую указателем `addrlen`, изначально записывается размер структуры.

Для идентификации соединения используется два адреса сокетов между которыми происходит обмен данными.

Установка соединения (клиент)

На стороне клиента для установления соединения используется функция `connect`, которая имеет следующий прототип:

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
int connect(int sockfd, struct sockaddr *serv_addr,
            int addrlen);
```

`sockfd` - сокет, который будет использоваться для обмена данными с сервером, `serv_addr` содержит указатель на структуру с адресом сервера, а `addrlen` - длину этой структуры. Обычно сокет не требуется предварительно привязывать к локальному адресу, так как функция `connect` сделает самостоятельно, подобрав подходящий свободный порт, при желании можно принудительно назначить клиентскому сокету некоторый номер порта, используя `bind` перед вызовом `connect`.

После того как соединение установлено, можно начинать обмен данными. Для этого используются функции `send` и `recv`. В Unix для работы с сокетами можно использовать также файловые функции `read` и `write`.

Функция `send` используется для отправки данных и имеет следующий прототип:

```
int send(int sockfd , const void *msg, int len , int
        flags );
```

`sockfd` - дескриптор сокета, через который отправляются данные. `msg` - указатель на буфер с данными, `len` - длина буфера в байтах, а `flags` - набор битовых флагов, управляющих работой функции.

Функция `send` возвращает число байтов, которое на самом деле было отправлено (или `-1` в случае ошибки). Это число может быть меньше указанного размера буфера. Если нужно отправить весь буфер целиком, то нужно написать многократный вызов этой функции:

```
int sendall(int s, char *buf, int len, int flags)
{
    int total = 0;
    int n;

    while(total < len)
    {
        n = send(s, buf+total, len-total, flags);
        if(n == -1) { break; }
        total += n;
    }

    return (n == -1 ? -1 : total);
}
```

Для чтения данных из сокета используется функция `recv`:

```
int recv(int sockfd , void *buf , int len , int flags)  
    ;
```

Её использование аналогично `send`, она принимает дескриптор сокета, указатель на буфер и набор флагов. Возвращает количество прочитанных байтов, либо 0 если соединение было разорвано.

Заккрытие сокета

Закончив обмен данными, необходимо закрыть сокет с помощью функции `close`:

```
#include <unistd.h>
```

```
int close(int fd);
```

Либо можно запретить передачу в каком-то направлении:

```
int shutdown(int sockfd, int how);
```

Параметр `how` может принимать одно из следующих значений:

- 0 – запретить чтение из сокета
- 1 – запретить запись в сокет
- 2 – запретить и то и другое

Если в процессе обмена пакетами возникли ошибки, то все функции возвращают -1, записывая в глобальную переменную `errno` код ошибки. Получив ошибку можно либо вывести диагностическое сообщение при помощи функции `perror`, либо завершить программу.

Для отладки сетевого приложения, можно воспользоваться петлевым интерфейсом `loorbak`, которому соответствует константа `INADDR_LOOPBACK`.

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
char message[] = "Hello_tthere!\n";
char buf[sizeof(message)];
int main()
{
    int sock;
    struct sockaddr_in addr;
    sock = socket(AF_INET, SOCK_STREAM, 0);
    if(sock < 0)
    {
        perror("socket");
        exit(1);
    }
    addr.sin_family = AF_INET;
    addr.sin_port = htons(3000); // или любой
                               другой порт...
```

```
addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);  
if (connect(sock, (struct sockaddr *)&addr,  
           sizeof(addr)) < 0)  
{  
    perror("connect");  
    exit(2);  
}  
send(sock, message, sizeof(message), 0);  
recv(sock, buf, sizeof(message), 0);  
printf(buf);  
close(sock);  
return 0;  
}
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
int main()
{
    int sock, listener;
    struct sockaddr_in addr;
    char buf[1024];
    int bytes_read;
    listener = socket(AF_INET, SOCK_STREAM, 0);
    if(listener < 0)
    {
        perror("socket");
        exit(1);
    }
    addr.sin_family = AF_INET;
    addr.sin_port = htons(3000);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

```
if (bind(listener, (struct sockaddr *)&addr,
    sizeof(addr)) < 0)
{
    perror("bind");
    exit(2);
}
listen(listener, 1);
while(1)
{
    sock = accept(listener, NULL, NULL);
    if(sock < 0)
    {
        perror("accept");
        exit(3);
    }
}
```

```
    while(1)
    {
        bytes_read = recv(sock, buf, 1024, 0);
        if(bytes_read <= 0) break;
        send(sock, buf, bytes_read, 0);
    }
    close(sock);
}
return 0;
}
```
